UNIVERSITÉ Sultan Moulay Slimane

Faculté des Sciences / Département d'Informatique

Filière / Master : M2 AIDC



TP 5 : AI Agentic

Implémentation de l'agent via LangChain

Réalisé par :

AHLAME LAOUAD
NASSIMA DOGHMI

Encadré par :

Pr. Lhoussaine Daoudi

Année Universitaire : 2026-2027

Date : Octobre 2026

1 Introduction

Dans cet atelier, on va explorer les bases de l'intelligence artificielle agentique tout en créant un agent avec langgraph et en traitant les différentes propriétés et fonctionnalités commençant de la réponse selon un contexte demandé, jusqu'à la gestion de mémoire pour élaborer une conversation harmonique et cohérente.

2 AI Agentique :

Avec l'évolution des solutions technologiques qui servent à améliorer la qualité de vie des êtres humains, une branche de l' IA est apparu c'est " l'IA agentique".

Qu'est ce que l'IA agentique ?

L'IA agentique est un système d'intelligence artificielle capable d'atteindre un objectif précis avec une supervision limitée. Il se compose d'agents IA, de modèles de machine learning capables d'imiter la prise de décision humaine pour résoudre des problèmes en temps réel. Dans un système multi-agent, chaque agent exécute une tâche spécifique et leurs actions sont coordonnées grâce à l'orchestration de l'IA. Au contraire des agents traditionnel qui suivent des règles codées et prédéfinies.

3 Réalisation de l'atelier :

3.1 Préparation de l'environnement et importation :

Pour appliquer et concevoir les propriétés et capacités de notre agent simple . On doit mettre en place un environnement de base cohérent :

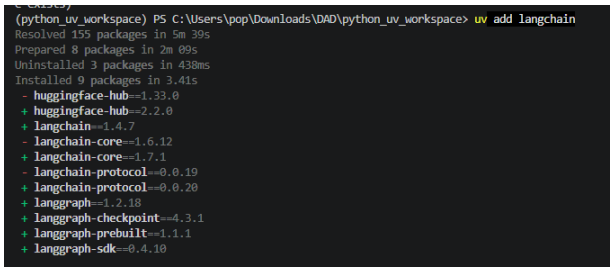


FIGURE 1 – Installation Langchain

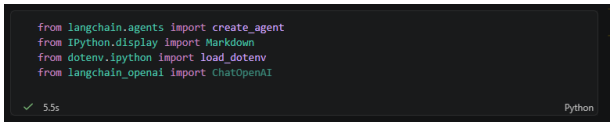


FIGURE 2 – Les imporations nécessaires



FIGURE 3 – Importer les variables d'environnement

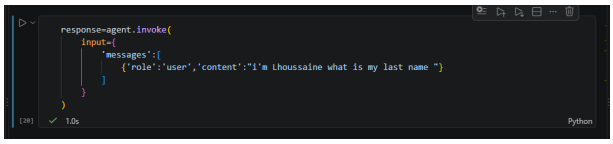


FIGURE 7 – Changement du contexte de réponse

3.2 Implémentation Agent Simple :

Dan cette partie, on veut mettre en place un agent simple sans mémoire ou tools pour examiner la qualité d'interaction avec demande utilisateur :



FIGURE 4 – Création agent simple

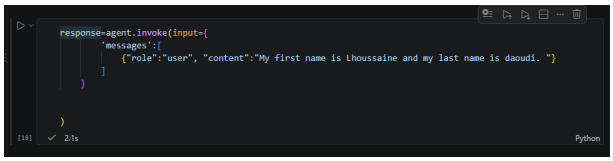


FIGURE 5 – Mise en place de contexte réponse

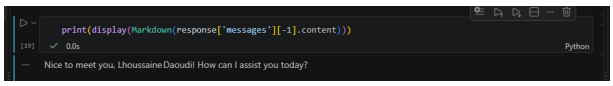


FIGURE 6 – Réponse de l'agent simple

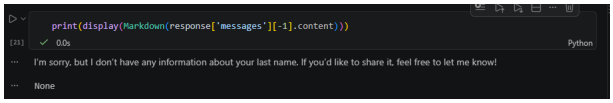


FIGURE 8 – Réponse de l'agent au contexte 2

Lors de ce test nous avons bien constaté que l'agent simple sans mémoire même si on a déjà lui mentionné le nom complet, il n'arrive pas à ce rappelait de ça si on a changé la question utilisateur.

3.3 Implémentation agent Dynamqie via langchain :

Pour manipuler le contexte et les réponses d'un agent dynamique on a utilisé une fonction de changment de résultat selon environnement choisi :

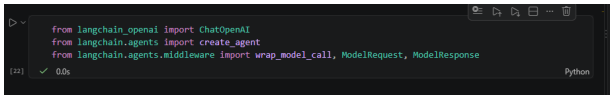


FIGURE 9 – les importations



FIGURE 12 – le modèle basique

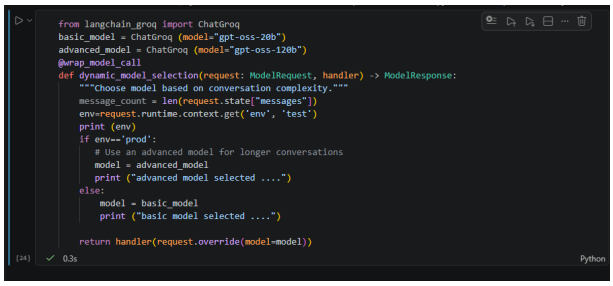


FIGURE 10 – Modèle dynamique selon l'env choisi



FIGURE 11 – création de l'agent dynamique



FIGURE 13 – Modèle avancé

Après le test de cette fonctionnalité de dynamicité, on doit examiner le rôle de la mémoire dans la gestion des conversation et la mise en place de contexte conversationnel cohérent.

3.4 Implémentation agent à mémoire :



FIGURE 14 – Les imports



FIGURE 15 – Création d'agent à mémoire volatile



FIGURE 16 – Réponse agent mémoire



FIGURE 17 – Réponse agent mémoire 2

Maintenat on va lier l'agent avec la mémoire postgresql :

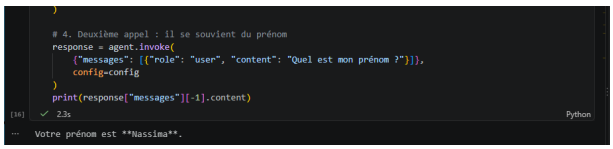


FIGURE 18 – Réponse d'agent en liaisant avec postgresql

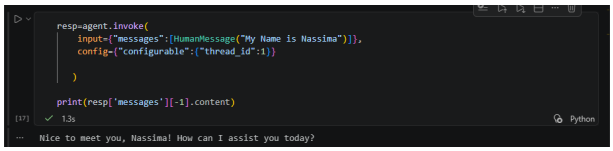


FIGURE 19 – réponse appel agent

3.5 Manipulation des outils agent :

Le rôle des outils pour les agents :

Les outils donnent aux agents la capacité d'agir. Les agents vont au-delà du simple rattachement d'outils au modèle en facilitant :

- **Plusieurs appels d'outils en séquence :** Déclenchés par un seul prompt.
- **Des appels d'outils en parallèle :** Lorsque cela est pertinent.
- **La sélection dynamique des outils :** En fonction des résultats précédents.
- **La logique de nouvelle tentative (retry) :** Ainsi que la gestion des erreurs.
- **La persistance de l'état :** Conservation des données entre les appels d'outils.

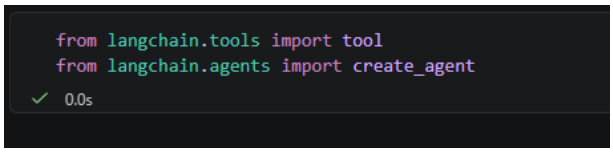


FIGURE 20 – Importation



FIGURE 25 – affichage résultat de nouvelles



FIGURE 21 – les outils de notre agent



FIGURE 22 – Création agent pour ajouter les outils



FIGURE 23 – affichage résultat appel tool get weather

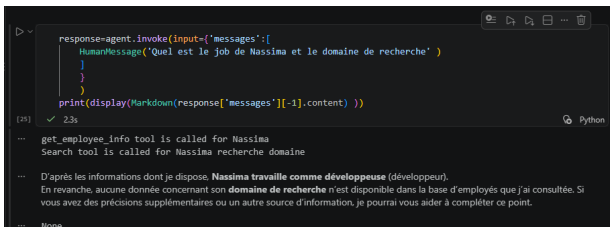


FIGURE 24 – Affichage résultat appel tool get salary

3.6 Utilisation de duckduck search :

L'outil duckduck search permet de faire la recherche web .



FIGURE 26 – Installation duckduck search

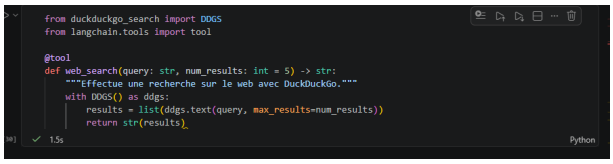


FIGURE 27 – Définition d'outil recherche web



FIGURE 28 – Choix du modele

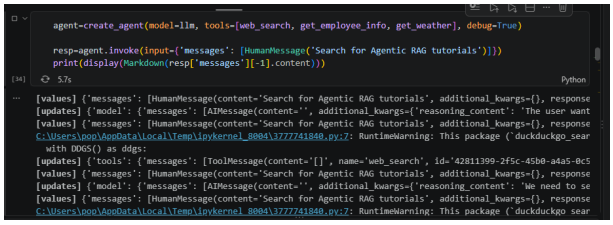


FIGURE 29 – Résultat de recherche duckduck

3.7 Utilisation tavily search :

Tavily est un outil qui permet à un agent de faire des recherches dans le web et retourner des résultats bien organisées.



FIGURE 30 – Installation tavily

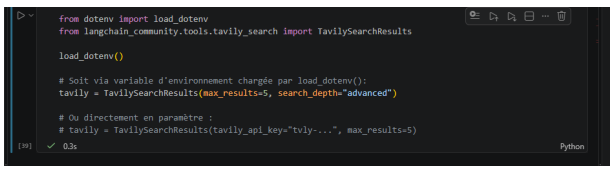


FIGURE 31 – Appel Tavily



FIGURE 35 – Résultat utilisation repl

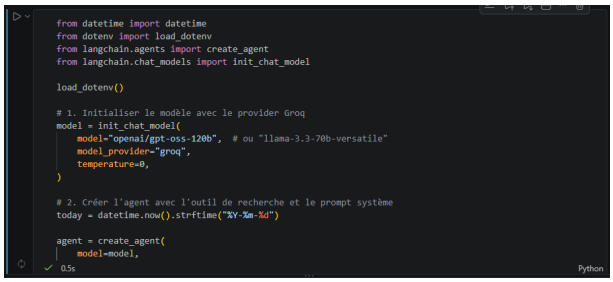


FIGURE 32 – Initialisation du modèle

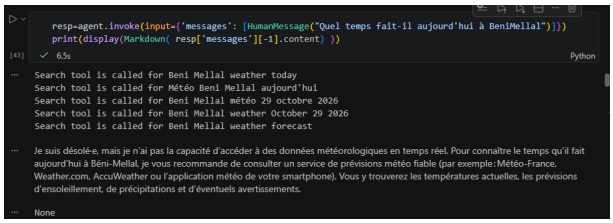


FIGURE 33 – Réponse agent Tavily

3.8 PythonREPL Tool :

PythonRepl est un outil qui permet à un agent AI d'executer un code python et de voir le résultat. On peut l'utiliser pour les calculs mathématiques, analyse des données et même pour les opérations statistiques.

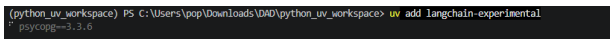


FIGURE 34 – Installation package

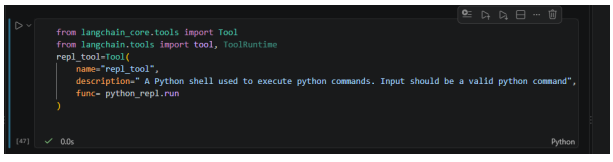


FIGURE 36 – Ajout tool repl

H



FIGURE 37 – Appel tool

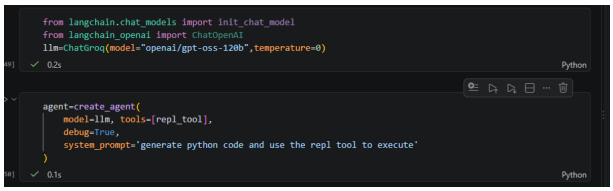


FIGURE 38 – Appel et création agent



FIGURE 39 – Execution du code et affichage du résultat



FIGURE 40 – essai d'un autre code



FIGURE 41 – Résultat du code

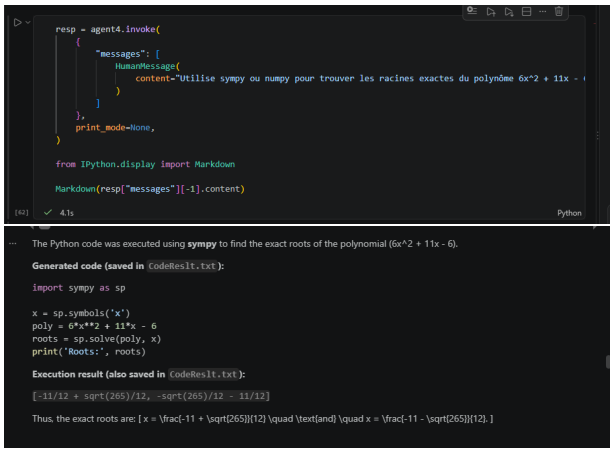


FIGURE 42 – autre test et son résultat

3.9 Tool Handling :

Est un outil qui permet de traiter et manager différentes demandes ou erreurs.

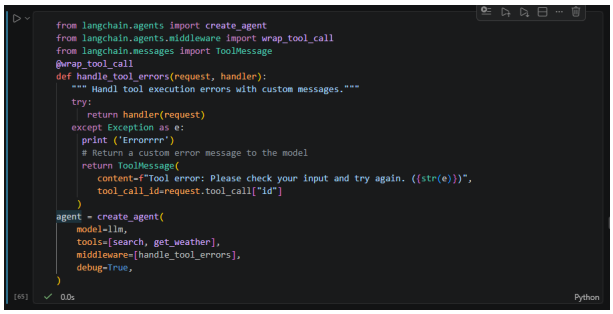


FIGURE 43 – handler des erreurs

3.10 Contrôle de système prompt :

Un système prompt est une méthode qu'on utilise pour bien définir le comportement d'un agent AI qu'est ce qu'il est permis ou non et comment réagir .



FIGURE 44 – Mise en place du système prompt



FIGURE 45 – Création caractéristiques agent



FIGURE 46 – Examination agent

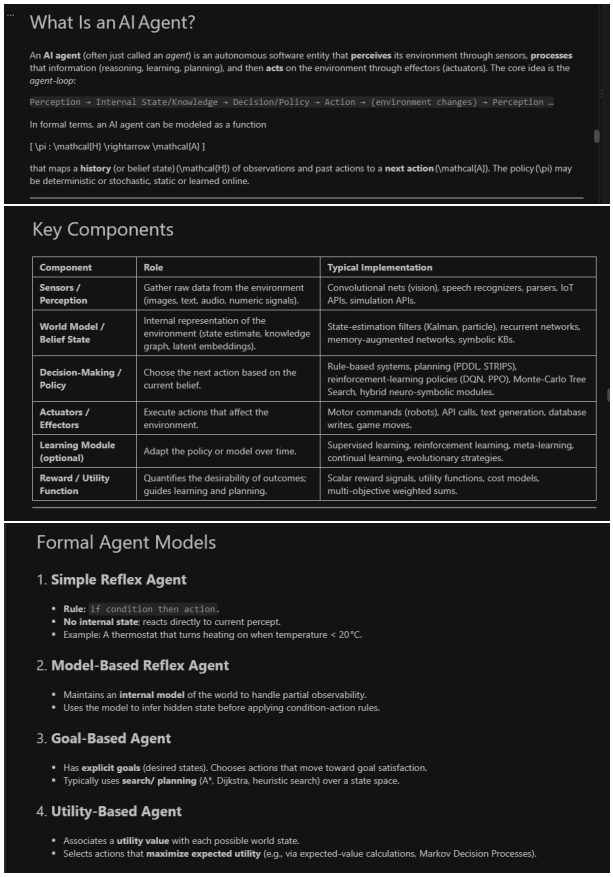


FIGURE 47 – réponse agent

3.11 Réponse structurée :



FIGURE 48 – organisation de la structure de la réponse

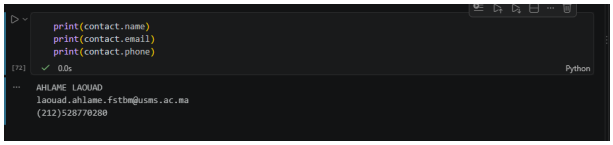


FIGURE 49 – Affichage

3.12 Custom Middleware :

Est une méthode dont on ajoute une couche entre l'agent AI et les opérations qu'il exécute, pour contrôler la façon de réalisation des tâches et ajouter des comportements cohérentes à notre système.

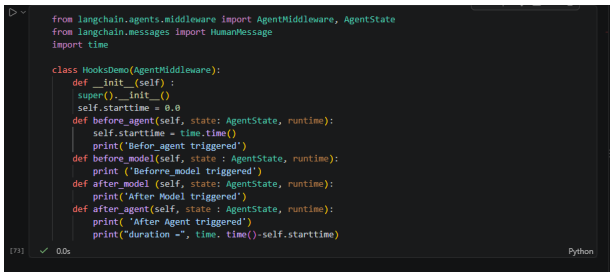


FIGURE 50 – Hook demo

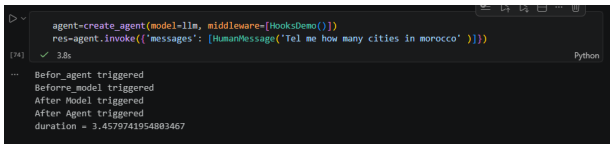


FIGURE 51 – Comportement agent

3.13 Utilisation de Rag comme outil :

Pour utiliser le traitement rag, on installé l'embedding de hugging face :

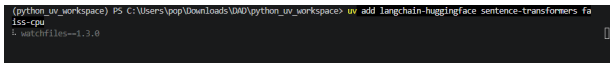


FIGURE 52 – Installation outil embedding

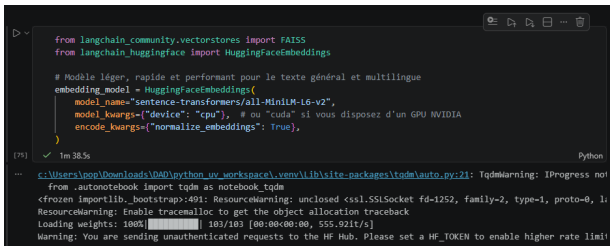


FIGURE 53 – Appel du modèle

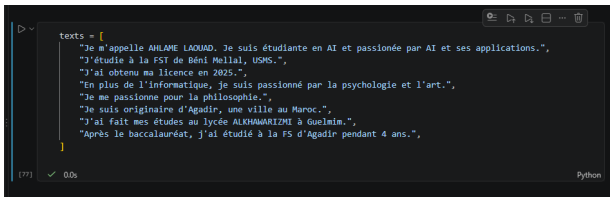


FIGURE 54 – Texte d'entrée



FIGURE 55 – Phase de recherche

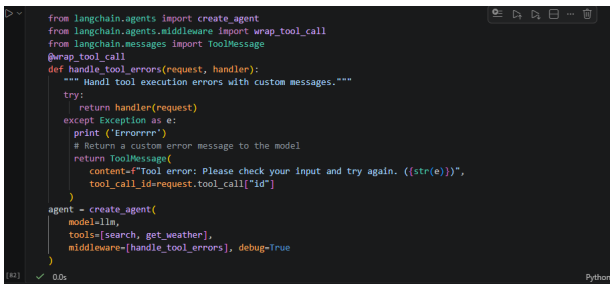


FIGURE 56 – définition d'outil

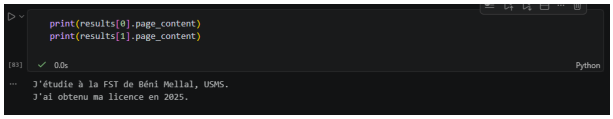


FIGURE 57 – Résultat

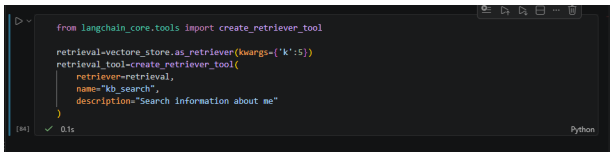


FIGURE 58 – Création d'outil retrieval



FIGURE 59 – Mise en place d'agent de recherche

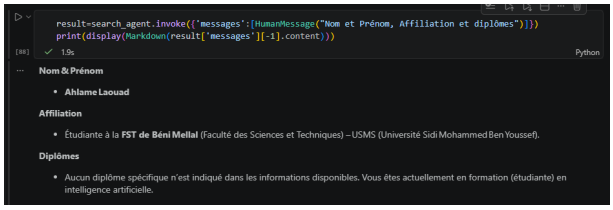


FIGURE 60 – Résultat de la recherche

3.14 Gardes feu intégrés :

Les gardes feu sont des outiles utiliser pour garder et contrôler le comporteent d'agent dans des limites aux différents comportements non voulu.

- **Input Guardrails :** Surveillance des entrées utilisateur avant leur transmission au LLM.
- **Output Guardrails :** Vérification des réponses avant leur transmission à l'utilisateur.
- **Safety Guardrails :** Prévention des contenus dangereux ou non autorisés.
- **Tool Guardrails :** Contrôle de l'utilisation des outils et blocage des opérations non autorisées.
- **Content Filtering :** Filtrage des contenus inappropriés.
- **PII Protection :** Protection des informations personnelles identifiables, telles que les numéros de téléphone et les mots de passe.

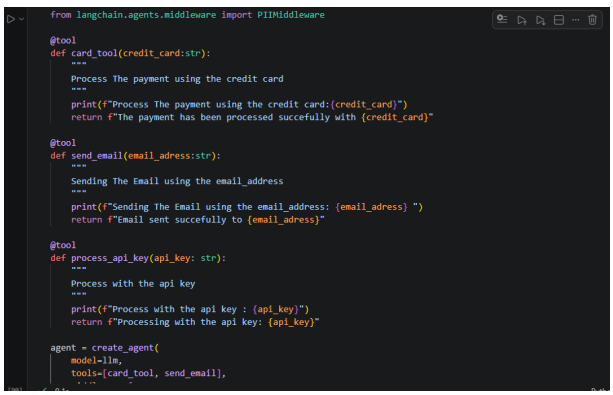


FIGURE 61 – Outils du garde

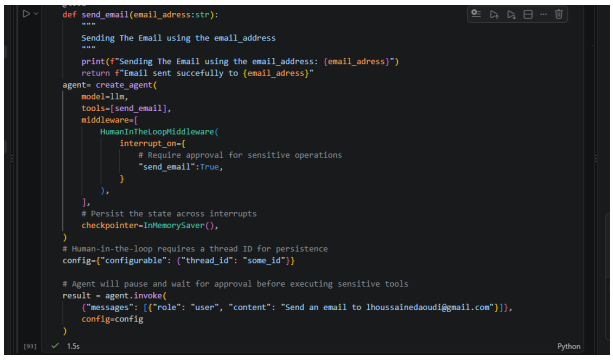


FIGURE 64 – HITL

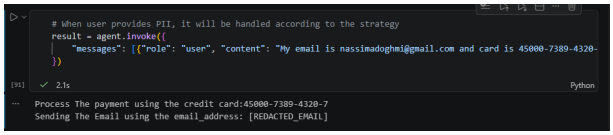


FIGURE 62 – Intégrattion PII données personnelles

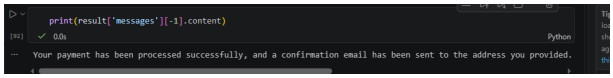


FIGURE 63 – Résultat PII

3.15 Human in The Loop :

La méthode de HITL, est une technique dans laquelle on intègre l'humain dans la prise des décisions et la réalisation des tâches.

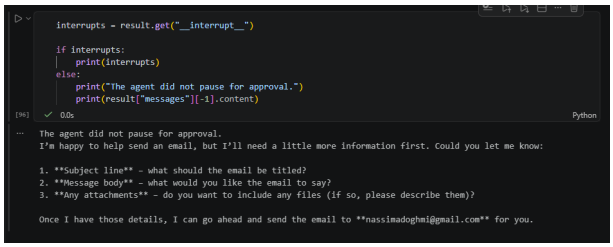


FIGURE 65 – Limitation d'agent dans la complétion de la tâche

l'utilisateur a donné l'accord alors voilà le résultat :

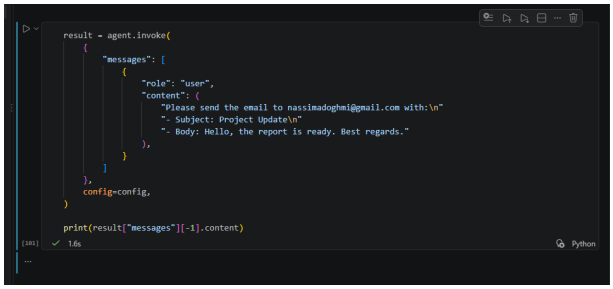


FIGURE 66 – tâche apprové par un humain

3.16 Garde fous personnalisés :

Dan cette partie on va tester le comportement d'un agent avec garde fous :

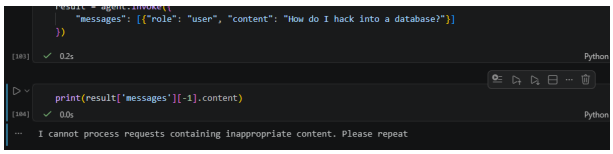


FIGURE 67 – Agent rejette la demande de pirater une base de données

Ici, on va tester un garde fous après l'agent :



FIGURE 68 – Detection et rejet d'une autre demande inapropriée

4 Conlusion :

Ce travail pratique a permis d'explorer de manière approfondie les concepts fondamentaux et avancés de l'**intelligence artificielle agentique** à l'aide des frameworks **LangChain** et **LangGraph**. En partant d'un agent conversationnel réactif sans état, nous avons progressivement mis en œuvre une architecture autonome capable de raisonner, de mémoriser et d'interagir avec divers environnements.

Plusieurs aspects clés ont été abordés et validés expérimentalement :

- **Gestion de la mémoire et persistance de l'état :** La mise en évidence des limites d'un agent sans état a justifié l'intégration de mécanismes de persistance, qu'ils soient volatils via la mémoire vive (`MemorySaver`) ou durables au moyen d'une base relationnelle (`PostgresSaver`). Ce suivi contextuel est indispensable pour assurer des interactions multi-tours cohérentes.
- **Outillage dynamique (*Tool Calling*) et recherche d'information :** L'interconnexion avec des outils externes a permis de lever les verrous cognitifs du modèle linguistique. Nous avons ainsi intégré avec succès des moteurs de recherche web (`DuckDuckGo`, `Tavily`) pour l'accès aux données en temps réel, l'environnement d'exécution de code `PythonREPL` (avec `SymPy`/`NumPy`) pour le calcul formel, ainsi qu'un pipeline **RAG** s'appuyant sur des plongements vectoriels Hugging Face et un index **FAISS**.
- **Contrôle, extensibilité et robustesse :** Grâce à l'injection dynamique de *system prompts* selon le profil utilisateur et à l'extraction de réponses structurées via des schémas de validation, le flux de traitement gagne en prévisibilité. Par ailleurs, le recours aux *middlewares* a facilité l'observabilité de l'agent (mesure du temps d'exécution) et la capture robuste des erreurs d'outils.
- **Sécurité et supervision humaine (*Human-in-the-Loop*) :** L'implémentation de garde-fous (*Guardrails*) et de filtres de données personnelles (PII) a démontré la possibilité de confiner l'agent à un périmètre strict. Enfin, l'introduction du contrôle humain (*HITL*) lors d'actions

critiques garantit une gouvernance éthique et sécurisée indispensable en environnement opérationnel.

En conclusion, ce TP met en exergue la transition entre les modèles de langage génératifs passifs et des systèmes agentiques décisionnels et coordonnés. Ce travail constitue un socle solide pour le développement futur de systèmes multi-agents distribués et spécialisés.